

MLP - Project 1 Documentation

Tomáš Majerčík, Damián Štyrák

Introduction to Neural Networks

Faculty of Informatics and Information Technologies, STU

November 9, 2025

Contents

1	Introduction	2
2	Dataset Analysis	2
3	Data Preprocessing and Normalization	2
4	Model Configuration	3
5	Experiment Tracking with W&B	3
5.1	Examples	3
5.1.1	Normal Training Behavior	3
5.1.2	Unstable or broken training	4
5.1.3	Overfitting training	5
5.2	Confusion matrix	5
6	Hyperparameter Optimization	5
6.1	Hyperparameter Optimization Results	6
7	Results	7
7.1	The best model test results:	7
8	Conclusion	7
A	Appendix	8
A.1	W&B Project and Experiment Logs	8
A.2	Configuration Files	8
A.2.1	best.yaml	8
A.2.2	sweep.yaml	9

1 Introduction

The task was to analyze the Bioresponse dataset, a binary classification problem, and to develop a Multi-Layer Perceptron (MLP) model capable of accurately predicting the target variable. The project involved applying improvement techniques such as dropout regularization and batch normalization to improve model generalization and stability.

The main goal was to minimize validation loss while maximizing performance metrics such as accuracy and F1-score. A key challenge was to prevent the model from overfitting on the training data. All experiments were tracked and logged using Weights & Biases (W&B), and a hyperparameter sweep was conducted to identify the optimal model configuration.

2 Dataset Analysis

For this project, we used the Bioresponse dataset from OpenML. It contains data about different molecules and whether they triggered a biological response or not. Each molecule is described by 1,776 numerical features (named D1 to D1776), and the target column tells us if the response was observed (1) or not (0). There are 3,751 samples in total, and all the feature values are already normalized between 0 and 1, which is great because there is no need to scale them again.

We started by loading the dataset and checking its structure. There were no missing values or duplicate rows, and the target classes were fairly balanced - a bit more ones than zeros, but nothing too extreme.

3 Data Preprocessing and Normalization

Since the values were already normalized, we did not need to do any extra preprocessing like outlier removal or scaling.

Next, we checked the distribution of the features using a normality test. None of the features were normally distributed, but that is okay because our task is classification, not regression, so normality is not required. To reduce the number of features and avoid redundancy, we calculated the correlation matrix and removed features that were highly correlated (above 0.9). Reducing correlated features can help improve model performance and training speed.

After that, we used mutual information to measure how strongly each feature is related to the target variable. Mutual information is a concept from information theory that tells us how much knowing the value of one variable helps us predict the value of another. In our case, it shows how much information a feature gives us about whether the molecule triggered a response. Unlike correlation, mutual information can detect both linear and nonlinear relationships, which makes it useful when the data doesn't follow a normal distribution. Features with higher mutual information scores are considered more relevant for prediction.

We used an 80%–10%–10% data split to provide enough samples for training while reserving separate validation and test sets for unbiased model evaluation, while keeping 33 features from original 1751.

4 Model Configuration

The MLP model was implemented using PyTorch as a modular and configurable architecture, defined dynamically through external YAML configuration files. This approach enabled flexible control over network structure, activation functions, dropout, normalization, and training parameters without changing the source code. The design supported easy experimentation, reproducibility, and seamless integration with W&B sweeps for automated hyperparameter tuning.

5 Experiment Tracking with W&B

Weights & Biases (W&B) is a platform for experiment tracking and visualization used to monitor model performance, compare runs, and manage hyperparameter optimization. It provides a web dashboard where metrics such as loss, accuracy, and F1-score can be visualized in real time, making it easier to analyze model behavior during training.

In this project, W&B was used to log training and validation metrics, model configurations, and hyperparameter sweeps. Each training run was recorded automatically, allowing side-by-side comparison of results. The combined graphs of training and validation loss helped detect overfitting - when the training loss continued to decrease while validation loss increased. Similarly, by tracking accuracy and F1-score trends, we could evaluate model generalization across different configurations.

During training, we tracked:

- **Loss (training and validation):** used to monitor optimization progress and detect overfitting, where validation loss stops improving while training loss continues to decrease.
- **Accuracy:** provided a straightforward measure of overall correctness across predictions.
- **F1-score:** chosen as a balanced metric between precision and recall, giving a more reliable view of model performance on imbalanced data.
- **Epoch:** number of epochs.

Together, these metrics allowed clear evaluation of both learning efficiency and model generalization.

5.1 Examples

5.1.1 Normal Training Behavior

The following plots illustrate a stable training process for one of the well-performing models. Both training and validation loss decrease smoothly over epochs, while valida-

tion accuracy and F1-score steadily improve and stabilize.

This indicates that the model is learning effectively without overfitting. The curves show consistent convergence, and early stopping would likely prevent further unnecessary training.

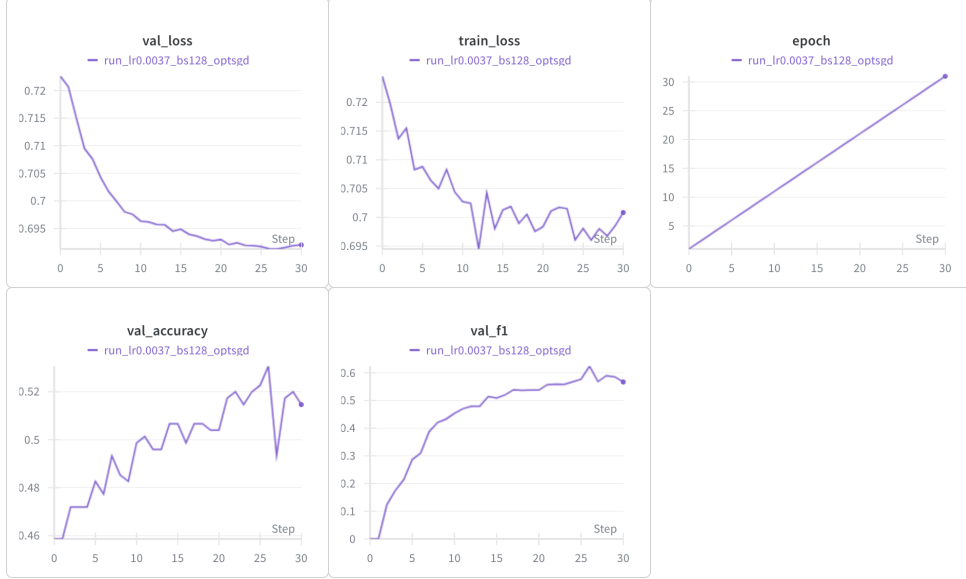


Figure 1: Normal Training Behavior Graphs

5.1.2 Unstable or broken training

This run demonstrates unstable learning behavior. The loss stagnates early, and both validation accuracy and F1-score oscillate between extremes, indicating that the model fails to converge.

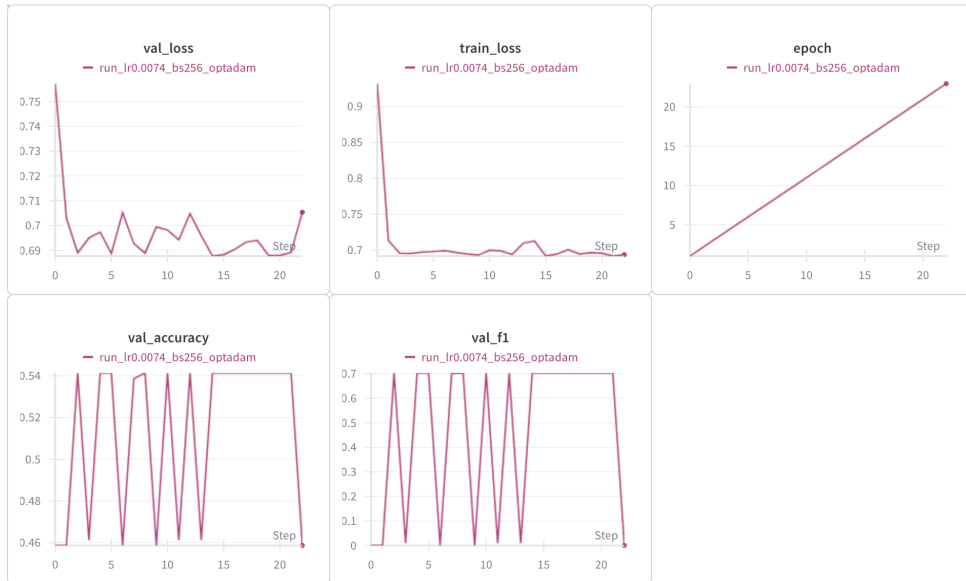


Figure 2: Unstable or broken training Graphs

5.1.3 Overfitting training

This training run shows early signs of overfitting. The training loss continues to decrease steadily, while the validation loss reaches its minimum after only a few epochs and then starts to rise again. This divergence between the two curves indicates that the model begins to memorize training data rather than improving generalization.

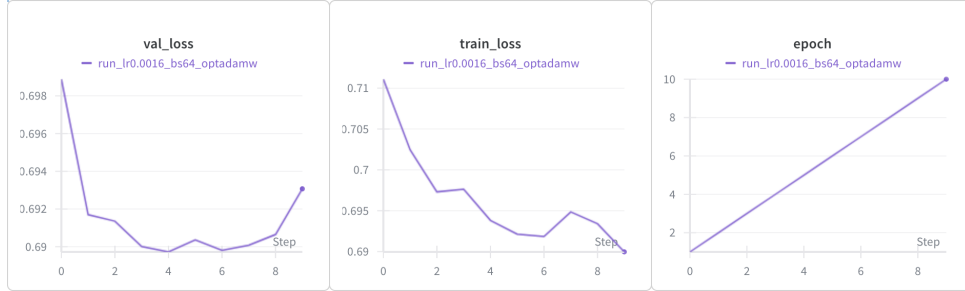


Figure 3: Overfitting training Graphs

5.2 Confusion matrix

The confusion matrix provides a clear overview of the model's classification performance by comparing predicted and true labels. It illustrates how well the model distinguishes between classes, revealing both correct predictions and misclassifications. From this visualization, it is possible to assess overall accuracy, detect potential class imbalance issues, and identify whether the model tends to favor one class over another.

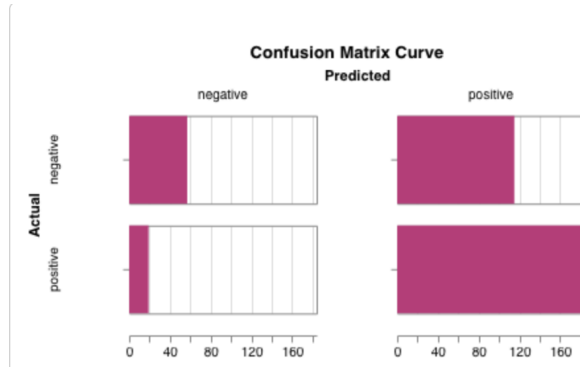


Figure 4: Confusion matrix

6 Hyperparameter Optimization

For hyperparameter optimization, we used a random sweep approach. This method was chosen because it provides a good balance between exploration and efficiency, allowing us to sample a diverse set of configurations without exhaustively testing every possible combination.

The sweep aimed to evaluate how different learning rates, batch sizes, optimizers, network depths, activation functions, dropout rates, and batch normalization settings influence model performance. We also included early stopping parameters to analyze how training stability and overfitting prevention vary under different conditions.

6.1 Hyperparameter Optimization Results

The scatter plot shows the distribution of validation F1-scores across all training runs, allowing quick identification of both high- and low-performing configurations. And the parameter importance chart highlights which hyperparameters had the strongest influence on model performance.

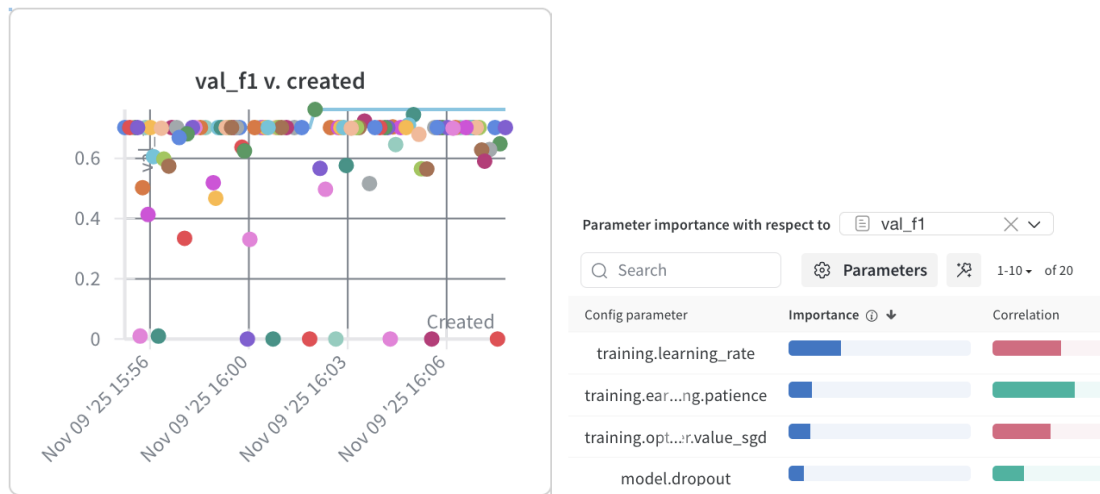


Figure 5: Scatter plot and Parameter importance chart

The parallel coordinates plot displays each training run as a line passing through axes representing different hyperparameters and the resulting performance metric. By observing where the highest-performing lines cluster, we can detect patterns and dependencies - for example, that certain optimizers perform better with specific learning rates or that smaller dropout values yield higher accuracy.

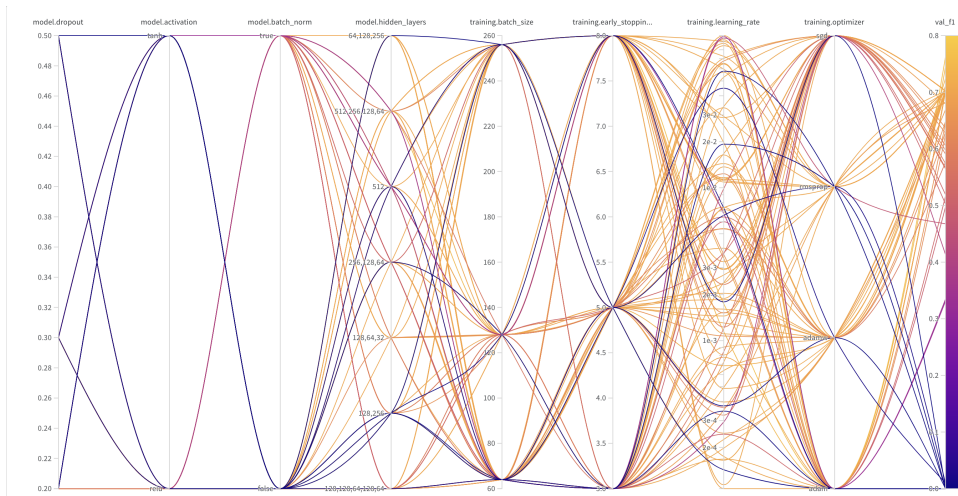


Figure 6: Correlation and parallel coordinates plot

Thanks to the hyperparameter sweep, we were able to identify and download the best-performing configuration directly from Weights & Biases. This setup achieved the highest validation F1-score and balanced training stability with generalization.

```
model.hidden_layers: [64, 128]
model.activation: tanh
model.dropout: 0.2
model.batch_norm: true
training.learning_rate: 0.0017417310784954333
training.optimizer: adam
training.batch_size: 64
training.early_stopping.patience: 8
```

7 Results

The final model, trained with the optimized hyperparameters from the W&B sweep, demonstrated stable learning and strong generalization. Throughout training, the validation accuracy improved to 0.75, indicating effective convergence and reduced overfitting.

The final evaluation on the held-out test set achieved an accuracy of 0.7543 and a loss of 0.0093, confirming that the tuned MLP model successfully captured the underlying patterns in the dataset while maintaining robustness across unseen data.

7.1 The best model test results:

```
F1-score: 0.7543
Loss: 0.0093
Accuracy: 0.7314
Precision: 0.7488
Recall: 0.7598
ROC-AUC: 0.7574
Confusion Matrix: TP=155, TN=120, FP=52, FN=49
```

8 Conclusion

The project successfully demonstrated the complete process of building, tuning, and evaluating an MLP model for binary classification on the Bioresponse dataset. Using W&B sweeps, we efficiently explored a wide range of hyperparameter configurations and identified an optimal setup that achieved stable convergence and strong generalization.

The final model reached a test accuracy of 0.75, confirming that the chosen architecture and training setup effectively captured underlying patterns in the data. Batch normalization and balanced regularization proved especially important for stabilizing learning and preventing overfitting.

Although performance is limited by the small dataset size, the workflow - from pre-processing to hyperparameter optimization - provides a solid foundation for extending the analysis with larger models or more advanced techniques in the future.

Additional Information

- *Implementation details, preprocessing steps, and experiment configurations are available in the source code and W&B experiment report linked in the Appendix.*
- *The full source code and configuration files, can be found in the project repository: https://github.com/StyroD/zneus_1/.*
- *Sections of this report were refined using AI assistance for clarity and formatting.*

A Appendix

A.1 W&B Project and Experiment Logs

Training runs, metrics, and hyperparameter optimization results are publicly available in the Weights & Biases project report: <https://api.wandb.ai/links/xmajercik-fiit-stu/gkrx0b66>

A.2 Configuration Files

A.2.1 best.yaml

The configuration file defining the final optimized model parameters:

```
# contents of configs/best.yaml
model:
  # input_dim: # set dynamically based on dataset
  hidden_layers: [64, 128]
  activation: tanh
  dropout: 0.2
  batch_norm: true
  output_dim: 1          # binary classification

training:
  epochs: 40
  batch_size: 64
  learning_rate: 0.0017417310784954333
  optimizer: adam
  device: cuda
  early_stopping:
    patience: 8
    counter: 0

experiment:
  group: "sweep-01-best"
  name: "mlp_bioresponse_best"
  output_dir: "results/"
  log_interval: 10
```

A.2.2 sweep.yaml

The configuration file specifying the hyperparameter search space and sweep method:

```
# contents of sweep.yaml
method: random    # random / bayes / grid / hyperband
metric:
  name: val_f1
  goal: maximize

parameters:
  training.learning_rate:
    distribution: log_uniform_values
    min: 1e-4
    max: 1e-1
  training.batch_size:
    values: [64, 128, 256]
  training.epochs:
    value: 40
  training.optimizer:
    values: ["adam", "sgd", "adamw", "rmsprop"]
  training.early_stopping.patience:
    values: [3, 5, 8]
  training.early_stopping.counter:
    value: 0

  model.hidden_layers:
    values:
      - [512]
      - [128, 64, 32]
      - [256, 128, 64]
      - [64, 128, 256]
      - [128, 256]
      - [512, 256, 128, 64]
      - [128, 128, 64, 128, 64]
  model.activation:
    values: ["relu", "tanh"]
  model.dropout:
    values: [0.2, 0.3, 0.5]
  model.batch_norm:
    values: [true, false]
```